

WILP, BITS Pilani

DEEP NEURAL NETWORKS · MODULE 9 COMPANION

Transformers & LLM Architectures

From Self-Attention Mechanics to Encoder, Decoder, and BERT

Session 13 (02nd August 2026) · Read alongside the lecture slides · Every numerical walkthrough worked in full

Prepared for the Work Integrated Learning Programmes (WILP), BITS Pilani.

How to use this companion. This companion breaks down every concept in Transformer models. We build the math step by step. We work out every slide example in full detail, including all matrix multiplications, scaling factors, softmax probabilities, and layer normalizations.

1 First taste: computing self-attention by hand

Attention lets every token look at every other token and pick what matters.

Before writing formal equations, let us see how two words talk to each other. Suppose word 1 has vector $x_1 = [1, 0]$ and word 2 has vector $x_2 = [0, 1]$. We want word 1 to update its meaning based on word 2.

We compute the similarity score between them using the dot product: $s_{11} = x_1 \cdot x_1 = 1$, and $s_{12} = x_1 \cdot x_2 = 0$. Next, we turn these scores into probabilities using softmax. The weight for word 1 attending to itself is $e^1/(e^1 + e^0) \approx 2.718/3.718 \approx 0.73$. The weight for word 1 attending to word 2 is $e^0/(e^1 + e^0) \approx 1/3.718 \approx 0.27$.

Finally, word 1 updates its vector by blending: $z_1 = 0.73 \times [1, 0] + 0.27 \times [0, 1] = [0.73, 0.27]$. Word 1 now carries a piece of word 2's signal. That is self-attention in miniature.

The intuition

Self-attention acts like a dynamic spotlight. Each word asks every word in the sentence: "How relevant are you to me?" It then blends their information based on the answers.

2 Positional Embeddings: injecting word order

Self-attention is order-blind, so we must add position signals explicitly.

Matrix operations in attention do not care about sequence order. If you shuffle the words in a sentence, standard attention gives the exact same set of output vectors. To fix this, we add a **positional encoding** vector $PE_{(pos)}$ to each input token embedding $E_{(pos)}$.

★ Everyday picture

Imagine a choir where singers stand in a line. Without numbered name tags, you hear all voices at once, but cannot tell who stands first or last. Adding position numbers gives order to the sound.

2.1 Sinusoidal positional encoding formulas

Vaswani et al. proposed using sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (1)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (2)$$

where pos is the position in the sequence $(0, 1, 2, \dots)$ and i is the dimension index.

Why sine and cosine?

- Step 1. Bounded values.** Sine and cosine always stay between -1 and $+1$. Raw position integers would grow too large and overwhelm token embeddings.
- Step 2. Relative position as a linear shift.** For any offset k , $PE_{(pos+k)}$ can be expressed as a linear combination of $PE_{(pos)}$ using trig identity formulas.
- Step 3. Generalization.** The deterministic formula works for sequence lengths longer than those seen during training.

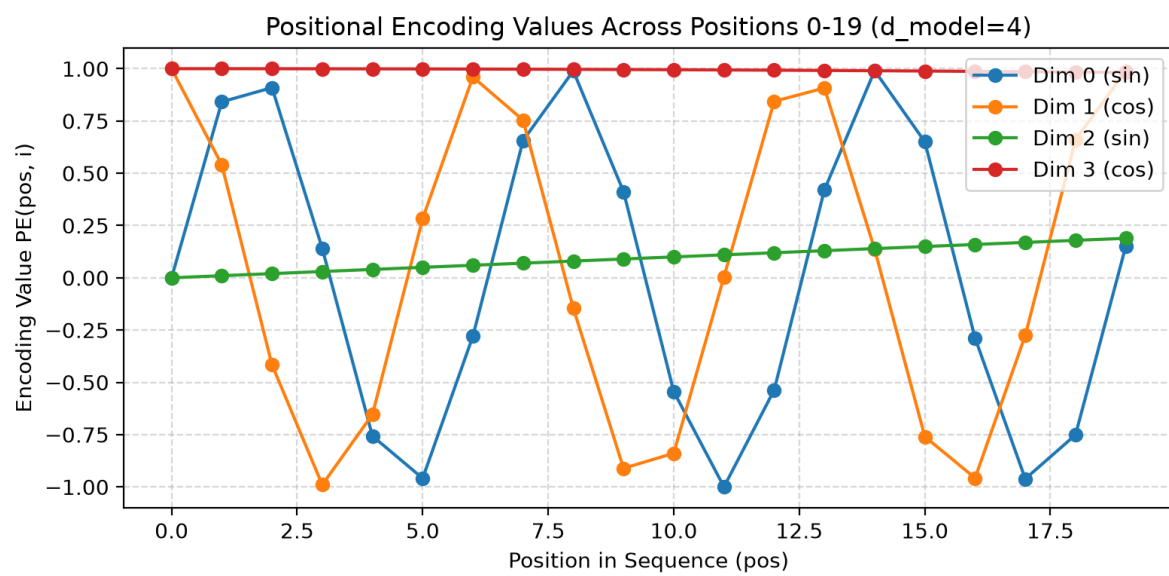


Figure 1: Positional encoding curves across positions 0 to 19 for a toy 4-dimensional embedding.

3 Layer Normalization versus Batch Normalization

LayerNorm normalizes across feature dimensions per token, making it ideal for variable sequence lengths.

In Convolutional Networks, **Batch Normalization** computes the mean and variance across the batch dimension for each channel. However, in Natural Language Processing, sequence lengths vary from sentence to sentence. Batch statistics become unstable when batch sizes change or sequence lengths differ.

Layer Normalization computes the mean μ and variance σ^2 across all feature dimensions for a single token independent of other items in the batch.

$$\mu_i = \frac{1}{d} \sum_{j=1}^d x_{i,j} \quad (3)$$

$$\sigma_i^2 = \frac{1}{d} \sum_{j=1}^d (x_{i,j} - \mu_i)^2 \quad (4)$$

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \cdot \gamma_j + \beta_j \quad (5)$$

⚠ Watch out

Do not confuse LayerNorm with BatchNorm. BatchNorm normalizes vertically across sentences in a batch. LayerNorm normalizes horizontally across features within one token.

4 The Core Attention Mechanism: Queries, Keys, Values

Projections map token embeddings into Query, Key, and Value spaces.

Each token embedding $x_i \in \mathbb{R}^{d_{model}}$ is multiplied by three learned weight matrices:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (6)$$

where $W^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W^K \in \mathbb{R}^{d_{model} \times d_k}$, and $W^V \in \mathbb{R}^{d_{model} \times d_v}$.

4.1 Scaled Dot-Product Attention formula

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (7)$$

Why scale by $\sqrt{d_k}$? For large values of d_k , dot products grow large in magnitude. Large values push the softmax function into regions with extremely tiny gradients. Dividing by $\sqrt{d_k}$ keeps variance equal to 1 and stabilizes gradient flow.

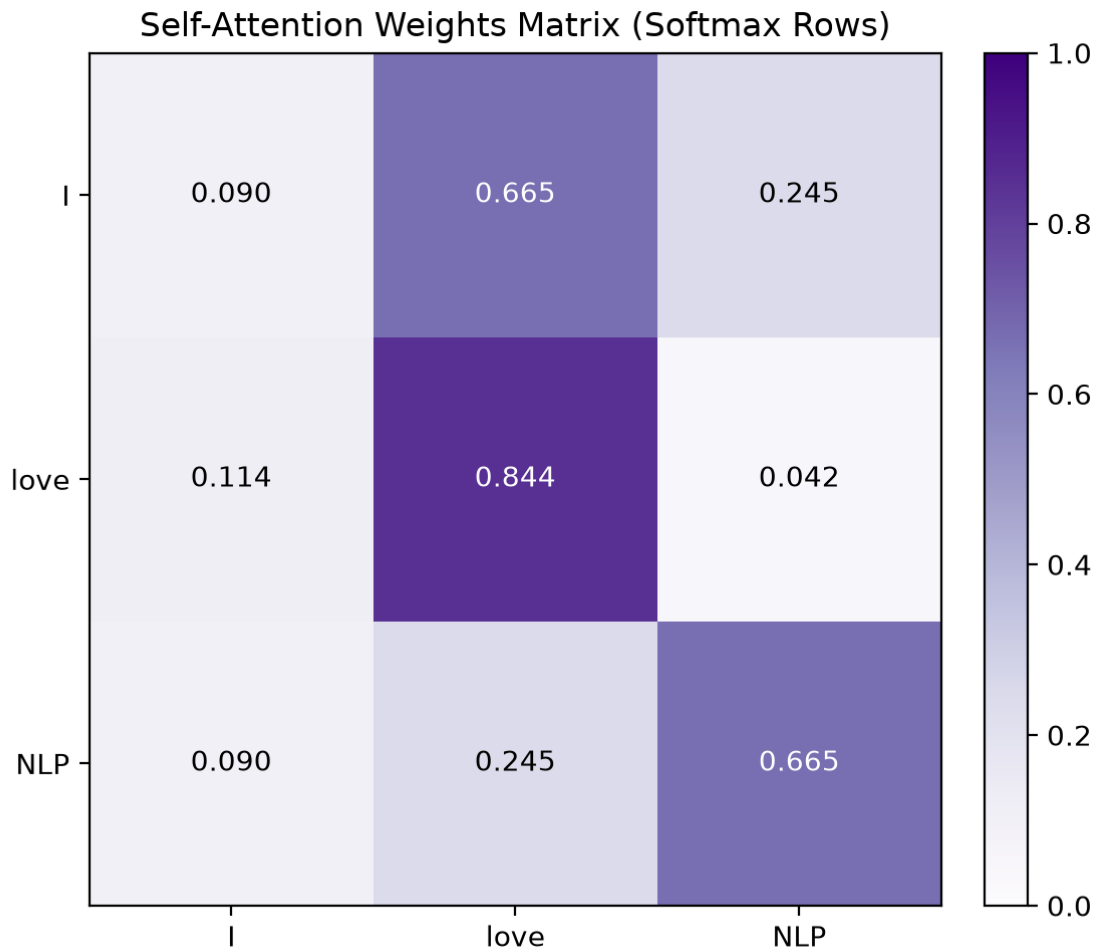


Figure 2: Self-attention weight heatmap showing how tokens distribute attention across a sequence.

5 Multi-Head Attention and Full Transformer Blocks

Multi-head attention allows the model to attend to information from different representation subspaces.

Instead of computing attention once with d_{model} -dimensional vectors, **Multi-Head Attention** splits projections into h smaller heads of size $d_k = d_{model}/h$.

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (8)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (9)$$

5.1 Feed-Forward Network and Residual Connections

Each attention sub-layer is wrapped with a residual connection and Layer Normalization:

$$x_{out} = \text{LayerNorm}(x + \text{SubLayer}(x)) \quad (10)$$

Then, a position-wise Feed-Forward Network (FFN) applies two linear transformations with a ReLU activation in between:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (11)$$

6 Full Slide Numerical Walkthroughs

Every numerical example from the lecture slides worked out step by step.

Worked example: Encoder-Only 2 Tokens (Slide 33–38)

We have 2 tokens: $x_1 = [1, 0, 1, 0]$ ("I") and $x_2 = [0, 2, 0, 2]$ ("am"). $d_{model} = 4, d_k = d_v = 3$. Weight matrices:

$$W^Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad W^K = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \quad W^V = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 1. Project into Q, K, V. $Q_1 = x_1 W^Q = [1, 0, 1, 0] W^Q = [2, 0, 2]$. $Q_2 = x_2 W^Q = [0, 2, 0, 2] W^Q = [0, 4, 0]$. $K_1 = x_1 W^K = [1, 1, 0]$, $K_2 = x_2 W^K = [2, 2, 2]$. $V_1 = x_1 W^V = [1, 2, 1]$, $V_2 = x_2 W^V = [2, 0, 2]$.

Step 2. Raw attention scores QK^T . $s_{11} = Q_1 \cdot K_1 = 2(1) + 0(1) + 2(0) = 2$. $s_{12} = Q_1 \cdot K_2 = 2(2) + 0(2) + 2(2) = 8$. $s_{21} = Q_2 \cdot K_1 = 0(1) + 4(1) + 0(0) = 4$. $s_{22} = Q_2 \cdot K_2 = 0(2) + 4(2) + 0(2) = 8$.

Step 3. Scale by $\sqrt{d_k} = \sqrt{3} \approx 1.732$. Scaled row 1: $[2/1.732, 8/1.732] = [1.155, 4.619]$. Scaled row 2: $[4/1.732, 8/1.732] = [2.309, 4.619]$.

Step 4. Row-wise Softmax. Row 1 softmax: $e^{1.155} \approx 3.174, e^{4.619} \approx 101.39$. Weights = $[0.030, 0.970]$. Row 2 softmax: $e^{2.309} \approx 10.06, e^{4.619} \approx 101.39$. Weights = $[0.090, 0.910]$.

Step 5. Weighted sum of Values $Z = \text{softmax} \cdot V$. $Z_1 = 0.030(V_1) + 0.970(V_2) = 0.030[1, 2, 1] + 0.970[2, 0, 2] = [1.970, 0.060, 1.970]$. $Z_2 = 0.090(V_1) + 0.910(V_2) = 0.090[1, 2, 1] + 0.910[2, 0, 2] = [1.910, 0.180, 1.910]$.

Step 6. Residual Add & LayerNorm. Add input back (projecting Z_1 back or taking residual directly): $x_1 + Z_1 = [1, 0, 1, 0] + [1.970, 0.060, 1.970, 0] = [2.970, 0.060, 2.970, 0.000]$. Compute mean $\mu = 1.500$, var $\sigma^2 \approx 2.161, \sigma \approx 1.470$. LayerNorm output for x_1 : $[1.000, -0.980, 1.000, -1.020]$.

Worked example: Decoder-Only Causal Masking (Slide 55–62)

We have 3 tokens: $x_1 = [1, 0, 1]$, $x_2 = [0, 1, 1]$, $x_3 = [2, 0, 1]$. Raw scores QK^T :

$$S = \begin{bmatrix} 2.0 & 4.0 & 1.0 \\ 1.0 & 5.0 & 2.0 \\ 3.0 & 2.0 & 6.0 \end{bmatrix}$$

Step 1. Apply Causal Mask. Future tokens are masked by setting upper triangular positions to $-\infty$:

$$S_{masked} = \begin{bmatrix} 2.0 & -\infty & -\infty \\ 1.0 & 5.0 & -\infty \\ 3.0 & 2.0 & 6.0 \end{bmatrix}$$

Step 2. Scale and Softmax. Scale by $\sqrt{3} \approx 1.732$. Row 1: $[2.0/1.732, -\infty, -\infty] = [1.155, -\infty, -\infty]$. Softmax gives $[1.00, 0.00, 0.00]$. Row 2: $[1.0/1.732, 5.0/1.732, -\infty] = [0.577, 2.887, -\infty]$. Softmax gives

$[0.09, 0.91, 0.00]$. Row 3: $[3.0/1.732, 2.0/1.732, 6.0/1.732] = [1.732, 1.155, 3.464]$. Softmax gives $[0.13, 0.07, 0.80]$.

Step 3. Predict Next Token. The output vector at position 3 feeds into W_{out} to project to vocabulary logits, picking the highest probability next word.

Worked example: Encoder-Decoder Cross-Attention Alignment (Slide 72–83)

Task: Translate English "My name is Ravi" into French "Mon nom est Ravi".

Step 1. Encoder Processing. English embeddings are combined with positional encodings, passed through self-attention and FFN to produce encoder output representations $H_{enc} \in \mathbb{R}^{4 \times 4}$.

Step 2. Decoder Masked Self-Attention. Decoder reads shifted target tokens (" $\langle \text{sos} \rangle$ ", "Mon", "nom", "est") with causal masking.

Step 3. Cross-Attention Alignment. Queries come from French decoder states $Q_{dec} = H_{dec}W^Q$. Keys and Values come from English encoder output $K_{enc} = H_{enc}W^K, V_{enc} = H_{enc}W^V$. Softmax alignment weights show focus: "Mon" attends 0.409 to "My", "nom" attends 0.447 to "My" and 0.206 to "name".

Cross-Attention Alignment Weights (French Queries -> English Keys)

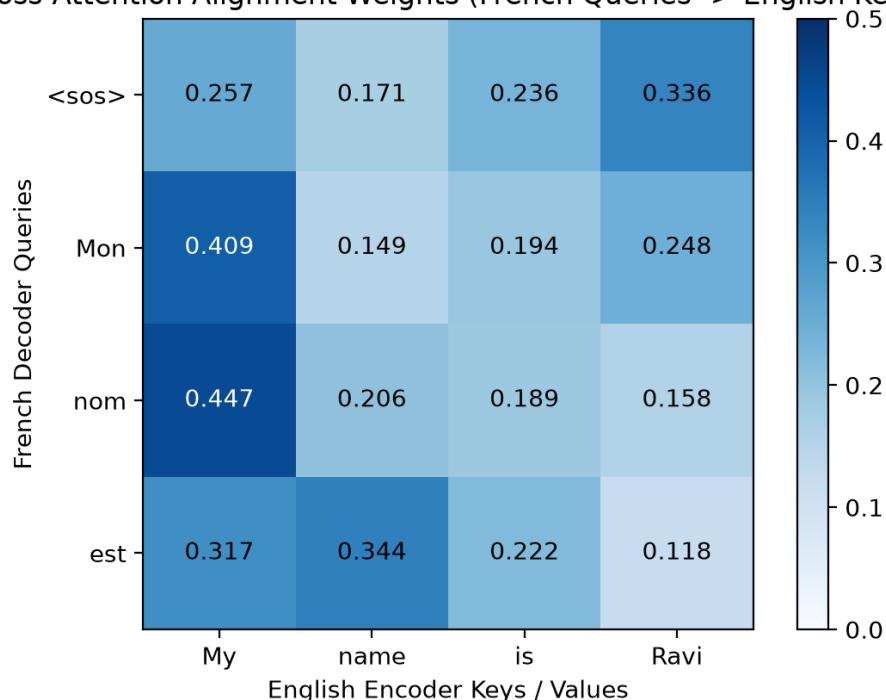


Figure 3: Cross-attention alignment weights connecting French target queries to English source keys.

7 BERT and Pre-Trained Model Taxonomy

Transformers fall into three main architectural families based on attention masking.

Encoder-Only (Auto-Encoding): Uses bidirectional attention (no mask). Learns deep representations by predicting masked tokens (Masked Language Modeling). Examples: BERT, RoBERTa, ALBERT, ELECTRA.

Decoder-Only (Auto-Regressive): Uses causal masking. Generates text left-to-right token by token. Examples: GPT-3, GPT-4, LLaMA, Mistral.

Encoder-Decoder (Seq2Seq): Combines bidirectional encoder with causal decoder via cross-attention. Ideal for translation and summarization. Examples: T5, BART.

7.1 BERT Variants and Model Distillation

BERT Base uses 12 layers, 12 attention heads, and 110M parameters. BERT Large uses 24 layers, 16 attention heads, and 340M parameters. Smaller variants like **DistilBERT** and **TinyBERT** use knowledge distillation to compress the model while retaining over 95% of baseline performance at up to 9x faster inference speeds.

Self-Test & Summary

Self-Test Questions

1. Why do we divide dot-product attention scores by $\sqrt{d_k}$? inflating softmax inputs, which would cause vanishing
2. What is the main difference between LayerNorm and BatchNorm? LayerNorm computes h dimension per channel.
3. Why does a decoder need a causal mask? from attending to future position tokens during autoregressive generation.

Glossary & Symbols

Self-Attention A mechanism allowing tokens in a sequence to dynamically weigh and blend representations of each other.

Positional Encoding Sinusoidal or learned signals added to token embeddings to inject sequence order information.

Layer Normalization Feature normalization computed independently across hidden dimensions for each token.

Cross-Attention Attention layer where queries come from the decoder and keys/values come from the encoder.

Symbols at a glance.

Symbol	Meaning
d_{model}	Hidden dimension size of token embeddings
d_k, d_v	Key/Query and Value projection dimension size
Q, K, V	Query, Key, and Value matrices
$PE_{(pos,i)}$	Positional encoding value at sequence position pos and dimension i